

Placement Interview Prep — Problem Set

DSA · OOPS · DBMS · Networking · SQL

Difficulty Tiers: Easy | Medium | Tough | Very Tough

Prepared for: Placement Operations & Employer Success

1. Data Structures & Algorithms (DSA)

EASY

1. Reverse an array/string in place.
2. Find the second largest element in an array without sorting.
3. Check if a string is a palindrome.
4. Find the missing number in an array of 1 to N.
5. Implement a stack using an array, and check for balanced parentheses.

MEDIUM

1. Find the first non-repeating character in a string.
2. Merge two sorted linked lists into one sorted list.
3. Find all pairs in an array that sum to a target value (two-pointer / hashing).
4. Detect a cycle in a linked list (Floyd's cycle detection).
5. Implement a queue using two stacks.
6. Find the maximum subarray sum (Kadane's Algorithm).
7. Level-order traversal of a binary tree (BFS).

TOUGH

1. Find the Kth largest element in an unsorted array in optimal time.
2. Detect and remove a loop in a linked list without extra space.
3. Find the lowest common ancestor (LCA) of two nodes in a Binary Search Tree.
4. Given a matrix, find the shortest path from top-left to bottom-right avoiding obstacles.
5. Implement a Trie and support insert/search/prefix search.
6. Find the longest palindromic substring in a given string.
7. Design and implement an LRU cache with $O(1)$ get/put.

VERY TOUGH

1. Given N jobs with deadlines and profits, find the maximum profit job sequence (Job Sequencing with constraints).
2. Solve the "Median of Two Sorted Arrays" problem in $O(\log(\min(m,n)))$ time.

3. Given a set of intervals representing meetings, find the minimum number of conference rooms required.
4. Implement a Segment Tree / Fenwick Tree supporting range sum queries and point updates.
5. Word Ladder: find the shortest transformation sequence from one word to another using a dictionary, changing one letter at a time.
6. Given a graph, detect if it contains a negative weight cycle (Bellman-Ford application).
7. Design a data structure that supports insert, delete, and getRandom in $O(1)$ average time.

2. Object-Oriented Programming (OOPS)

EASY

1. Define class vs. object with a real-world example.
2. Explain the four pillars of OOP (Encapsulation, Abstraction, Inheritance, Polymorphism) with one example each.
3. Write a class Employee with attributes and a method to display details.
4. What is a constructor? Differentiate between default and parameterized constructors.
5. What is the difference between a class and a structure?

MEDIUM

1. Explain method overloading vs. method overriding with code examples.
2. Implement runtime polymorphism using a base class Shape and derived classes Circle, Rectangle.
3. What is the difference between an abstract class and an interface? When would you use each?
4. Explain 'this' pointer / 'self' reference and its use cases.
5. What are access specifiers (public, private, protected)? Demonstrate encapsulation using getters/setters.
6. Explain the 'diamond problem' in multiple inheritance and how languages resolve it.

TOUGH

1. Design a class hierarchy for a Vehicle Rental System using inheritance and polymorphism.
2. Explain composition vs. inheritance — when should you prefer 'has-a' over 'is-a'?
3. Implement the Singleton design pattern and explain thread-safety concerns.
4. What is operator overloading? Implement it for a Complex Number class (addition, multiplication).
5. Explain virtual functions and vtables — how does dynamic dispatch work internally?
6. Design a system using the Factory design pattern for creating different types of notifications (Email, SMS, Push).

VERY TOUGH

1. Design a parking lot system using OOP principles (classes, relationships, extensibility for different vehicle types).
2. Explain and implement the Observer design pattern with a real-world pub-sub example.
3. Design an OOP-based system for a food delivery app covering Restaurant, Order, Delivery Partner, and Payment modules, focusing on SOLID principles.
4. Explain the SOLID principles with a violation example and a refactored solution for each.
5. How does garbage collection interact with object destruction in OOP languages? Discuss memory leaks caused by circular references.
6. Design an extensible plugin-based architecture using the Strategy and Decorator patterns together.

3. Database Management Systems (DBMS)

EASY

1. What is a Primary Key vs. a Foreign Key?
2. Define normalization and explain why it is needed.
3. What is the difference between DELETE, TRUNCATE, and DROP?
4. Explain the difference between a clustered and non-clustered index.
5. What is a schema? Differentiate between logical and physical schema.

MEDIUM

1. Explain 1NF, 2NF, and 3NF with an example table that violates each.
2. What is a transaction? Explain the ACID properties with examples.
3. Explain the difference between OLTP and OLAP systems.
4. What are joins? Explain INNER, LEFT, RIGHT, and FULL OUTER JOIN with examples.
5. What is a deadlock in DBMS? How can it be prevented?
6. Explain the difference between a view and a materialized view.

TOUGH

1. Explain BCNF (Boyce-Codd Normal Form) and how it differs from 3NF with an example.
2. What are the different levels of transaction isolation (Read Uncommitted to Serializable)? Explain dirty reads, non-repeatable reads, and phantom reads.
3. Explain how B+ Trees are used for indexing and why they're preferred over B-Trees in databases.
4. Design a normalized database schema (up to 3NF) for a University Management System (Students, Courses, Enrollments, Professors).
5. Explain the difference between optimistic and pessimistic concurrency control.
6. What is database sharding? Explain horizontal vs. vertical partitioning.

VERY TOUGH

1. Explain the internal working of query optimization — how does a query optimizer choose between a full table scan and an index scan?
2. Design a schema and explain the trade-offs for a multi-tenant SaaS application (shared schema vs. separate schema per tenant).
3. Explain distributed transactions and the Two-Phase Commit (2PC) protocol. What are its limitations?
4. How would you design a database to handle time-series data at scale (e.g., IoT sensor readings)?
5. Explain the CAP theorem and how it applies to choosing between SQL and NoSQL databases for a given use case.
6. Design a database recovery mechanism using write-ahead logging (WAL) and explain checkpointing.

4. Computer Networking

EASY

1. What is the difference between the OSI model and the TCP/IP model?
2. Define IP address vs. MAC address.
3. What is the difference between a Hub, Switch, and Router?
4. Explain the difference between TCP and UDP.
5. What is DNS, and why is it needed?

MEDIUM

1. Explain the three-way handshake in TCP connection establishment.
2. What is the difference between HTTP and HTTPS? How does SSL/TLS provide security?
3. Explain subnetting — given an IP and subnet mask, calculate the number of usable hosts.
4. What is NAT (Network Address Translation), and why is it used?
5. Explain the difference between symmetric and asymmetric encryption in the context of network security.
6. What happens when you type a URL into a browser and press Enter? (End-to-end flow.)

TOUGH

1. Explain congestion control mechanisms in TCP (slow start, congestion avoidance, fast retransmit).
2. What is the difference between a firewall, proxy server, and VPN in terms of network security?
3. Explain how routing protocols like OSPF and BGP determine the best path for data.
4. What is the difference between IPv4 and IPv6? Why was IPv6 introduced, and what challenges exist in migration?
5. Explain how load balancing works (Layer 4 vs. Layer 7) in a distributed system.
6. Explain the concept of a CDN (Content Delivery Network) and how it reduces latency.

VERY TOUGH

1. Design a network architecture for a company with multiple branch offices requiring secure, low-latency communication (consider VPN, MPLS, SD-WAN).
2. Explain how DDoS attacks work and outline mitigation strategies at different network layers.
3. Walk through how QUIC/HTTP3 differs from TCP-based HTTP/2 and why it reduces connection latency.
4. Explain how a distributed system achieves consensus over an unreliable network (e.g., Raft or Paxos) and why this matters for networked databases.
5. Design a solution for zero-downtime deployment across a globally distributed set of servers using DNS and load balancers.
6. Explain packet fragmentation and reassembly, and the issues it causes with Path MTU Discovery.

5. SQL

EASY

1. Write a query to fetch all employees earning more than 50,000 from an Employees table.
2. Write a query to find the total number of employees in each department.
3. Write a query to fetch distinct department names from the Employees table.
4. Write a query to sort employees by salary in descending order.
5. Write a query using LIKE to find all employees whose name starts with 'A'.

MEDIUM

1. Write a query to find the second highest salary in the Employees table without using LIMIT/TOP.
2. Write a query to find employees who earn more than the average salary of their department.
3. Write a query using GROUP BY and HAVING to find departments with more than 5 employees.
4. Write a query to find duplicate records in a table.
5. Write a query to fetch the names of employees who do not have a manager (self-join scenario).
6. Explain and write an example of a subquery vs. a correlated subquery.

TOUGH

1. Write a query using window functions (RANK(), DENSE_RANK(), ROW_NUMBER()) to find the top 3 highest-paid employees per department.
2. Write a query to find the running total (cumulative sum) of sales per month using window functions.
3. Write a query to find employees who joined in the same month as their manager.
4. Given an Orders table, write a query to find the month-over-month percentage growth in revenue.
5. Write a query to pivot row data into columns (e.g., converting monthly sales rows into a year-wise column format).
6. Write a query to identify gaps in a sequence of IDs in a table (missing invoice numbers).

VERY TOUGH

1. Write a recursive CTE (Common Table Expression) to find all subordinates under a given manager in an organizational hierarchy.
2. Given login/logout timestamp records, write a query to calculate the longest continuous session duration per user.
3. Write a query to detect and remove duplicate rows in a table without a primary key, keeping only one copy.
4. Design and write a query set to implement slowly changing dimensions (SCD Type 2) for a customer table in a data warehouse.
5. Write a query to find the three consecutive days with the highest total transaction amount from a transactions table.
6. Given two tables with overlapping date ranges (e.g., employee project assignments), write a query to detect all overlapping intervals.

Usage note: Each tier builds on the previous — Easy/Medium map well to screening rounds, Tough to technical interview rounds, and Very Tough to design/senior-level or R1-R2 rounds for product-based and GCC companies.